

Comparing Classical and Deep Learning Approaches for News Classification

MD.MASHFIQ UZ ZAMAN

Dept. of CSE
BRAC University
ID: 23101030

Mohammed Mashrekin Yakub

Dept. of CSE
BRAC University
ID: 23101253

Ayasha Islam

Dept. of CSE
BRAC University
ID: 19241002

Abstract—This paper presents a systematic empirical study of four-class news topic classification covering Business, Science and Technology, Sports, and World News. We experiment with 96,646 training samples and 12,000 balanced test samples across two word-representation paradigms — TF-IDF with SVD compression and Skip-gram Word2Vec embeddings — combined with three preprocessing regimes (none, extreme, optimum) and eight model architectures: Logistic Regression, a four-layer Deep Neural Network (DNN), SimpleRNN, GRU, LSTM, and their Bidirectional variants. A grid search over dropout rate $\{0.3, 0.5\}$, batch size $\{32, 64\}$, and epoch budget $\{5, 8\}$ produced 144 trained sequence-model checkpoints in addition to the classical baselines. The best validation configuration (Skip-gram, optimum preprocessing, GRU, units=128, dropout=0.3, batch=64, epochs=8) achieves 93.10% validation accuracy and 0.9183 macro F1. On the final held-out test set, the same setup yields 92.01% accuracy and 0.9200 macro F1. TF-IDF Logistic Regression (raw text, C=3) is the strongest classical baseline at 92.86% validation accuracy, barely surpassed by deep methods. Extreme preprocessing consistently degrades performance across all models, confirming that over-aggressive text reduction removes discriminative lexical features in this domain.

Index Terms—news classification, TF-IDF, Word2Vec, Skip-gram, GRU, LSTM, bidirectional RNN, text preprocessing, natural language processing, deep learning

I. INTRODUCTION

Automatic news topic classification is one of the most extensively studied problems in natural language processing (NLP). As online news consumption has grown exponentially, the ability to route, filter, and recommend articles at scale has become commercially and societally important. Manual labelling is both expensive and slow; robust automated classifiers are therefore a necessity in any large-scale news platform.

The dataset used in this project is an HTML-wrapped corpus of news items drawn from four topically distinct categories. Unlike clean benchmark datasets such as AG News [5], this corpus introduces several real-world challenges: pervasive HTML boilerplate, broken numeric character entities, wire-service attribution tags, a severe class imbalance in the training split (Sports at 39.75%). The principal contributions of this paper are:

- A detailed exploratory data analysis (EDA) characterising noise sources, class imbalance, length distributions, and source-tag patterns specific to this corpus.

TABLE I
DATASET OVERVIEW

Metric	Train	Test
Total rows	96,646	12,000
Missing text/label	0 / 0	0 / 0
Average word count	38.40	38.46
Median word count	38	38
Average char count	228.45	230.70
HTML tags (100%)	yes	yes
Broken entities (%)	33.30	31.96
Backslash artifacts (%)	10.27	10.67

- A controlled comparison of three preprocessing levels, isolating the effect of each cleaning decision on downstream model performance.
- An end-to-end benchmarking of eight model architectures across two representation paradigms with full hyperparameter tuning via grid search.
- An analysis of failure modes — in particular the instability of SimpleRNN and the consistent degradation caused by extreme preprocessing.
- A discussion of limitations and directions for future improvement, including transformer-based models.

The remainder of this paper is structured as follows. Section II describes the methodology including the dataset, EDA findings, preprocessing design, word representations, and model architectures. Section III presents the experimental results with tables and discussion. Section IV concludes with key takeaways, limitations, and future work.

II. METHODOLOGY

A. Dataset Description

The corpus is provided as two CSV files: `Training_data_13.csv` (96,646 rows) and `Test_data.csv` (12,000 rows). Each row contains two columns: *News Headline* (the raw text input) and *News Topic* (one of four string labels). Table I shows a high-level summary of the two splits.

B. Exploratory Data Analysis

EDA was conducted exclusively on the training split prior to any modelling decisions.

TABLE II
CLASS DISTRIBUTION

Class	Train N	Train %	Test N	Test %
Business	22,993	23.79	3,000	25.0
Science & Technology	23,487	24.30	3,000	25.0
Sports	38,421	39.75	3,000	25.0
World News	11,745	12.15	3,000	25.0
Total	96,646	100	12,000	100

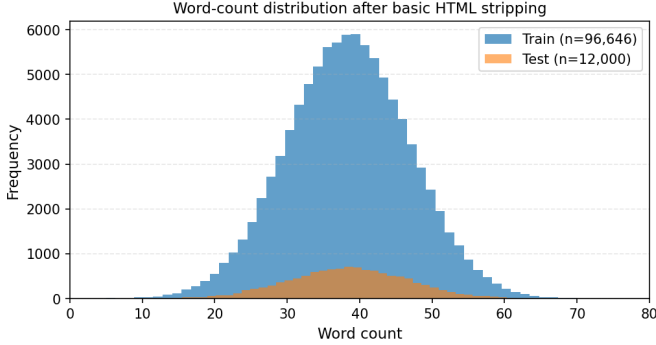


Fig. 1. Word-count distribution for training (blue) and test (orange) splits after basic HTML stripping.

1) *Class Distribution*: The training and test splits have fundamentally different class distributions, which is the single most important finding for evaluation design. The training set is heavily skewed toward Sports (39.75%) while World News accounts for only 12.15%. The test set is perfectly balanced at 25% per class. This mismatch means that raw accuracy, which weights classes by their frequency, is a misleading metric; macro-averaged F1 is the primary evaluation criterion throughout this work. The per-class counts are summarised in Table II.

2) *Artifact Analysis*: Every row in both splits is wrapped in `<html>` `<body>` tags and prefixed with the string `News Headlines:`, contributing no semantic content and up to 30 tokens of noise. Approximately 33.30% of training rows contain broken HTML-style numeric entities such as `#39;` (apostrophe), `quot;` (quotation mark), `amp;` (ampersand), or `#151;` (em-dash). A further 10.27% contain raw backslash characters preceding common words (e.g. `\said,` `\to`). Both artifact types must be normalised before any tokenisation step.

3) *Length and Lexical Statistics*: After stripping HTML, the average cleaned text length is 38.4 words and 228 characters — substantially longer than a typical news headline. This indicates that each record encodes a headline *plus* a short article summary. The word-length distribution is approximately Gaussian, with Sports having a slightly shorter median (37 words) than World News (39 words). Fig. 1 shows the distribution.

4) *Source-Wire Tags*: Named wire services appear parenthetically inside texts: Reuters, AP (Associated Press), and AFP (Agence France-Presse). Their frequency differs systematically by class — Reuters appears in 18.90% of Business

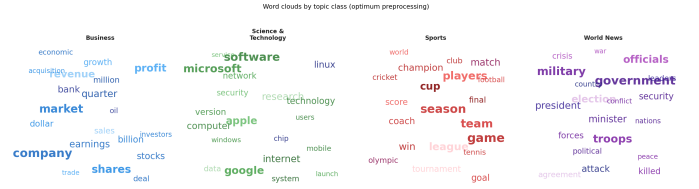


Fig. 2. Word clouds for each of the four topic classes after optimum preprocessing. Sports vocabulary (left) is dominated by player and team names; Business (centre-left) by financial terms; Science & Technology (centre-right) by product and research language; World News (right) by geopolitical terms.

rows and AFP in a disproportionate share of World News rows. These publisher-origin markers create a shortcut signal that models can exploit without learning genuine topic features. Removing them is therefore part of the optimum preprocessing pipeline.

C. Preprocessing Pipelines

Three text variants were produced and cached to disk for every split, enabling controlled ablation.

1) *No Preprocessing (none)*: The raw text is used exactly as provided, including HTML tags, the `News Headlines:` prefix, broken entities, and backslash artifacts. This variant serves as a baseline to quantify the value of cleaning.

2) *Extreme Preprocessing*: This variant applies the most aggressive cleaning: (1) HTML stripping via BeautifulSoup; (2) broken entity normalisation; (3) removal of the `News Headlines:` boilerplate; (4) source-wire tag removal; (5) lowercasing; (6) removal of all non-alphabetic characters (digits, punctuation, hyphens); (7) stopword removal using scikit-learn’s `ENGLISH_STOP_WORDS`; and (8) Porter stemming [10]. This pipeline reduces the average text length from 38.4 words to approximately 12–15 stems.

3) *Optimum Preprocessing*: Designed based on EDA insights, the optimum pipeline applies: (1) HTML stripping; (2) entity normalisation; (3) boilerplate removal; (4) source-wire tag removal; (5) lowercasing; (6) removal of non-alphanumeric characters excluding hyphens and apostrophes; and (7) whitespace normalisation. Numbers are *retained* because digits appear in 50% of Business and 52% of Sports rows and carry discriminative content (scores, financial figures, version numbers). Stopwords are *kept* because function-word patterns (e.g. “reports that”, “officials said”) differ across classes. No stemming is applied, preserving morphological distinctions. Table III summarises the three pipelines.

D. Validation Split Design

A 15% group-aware stratified hold-out was constructed as follows. A *group key* was defined as the lowercased, whitespace-normalised version of each raw text. All rows sharing a group key were assigned to the same partition. `train_test_split` was then applied at the group level (not the row level) with stratification on the label, yielding 82,240 training rows and 14,406 validation rows. Class weights were computed via

TABLE III
PREPROCESSING PIPELINE COMPARISON

Operation	None	Extreme	Optimum
HTML stripping			
Entity normalisation			
Boilerplate removal			
Source-tag removal			
Lowercasing			
Keep digits			
Keep stopwords			
Stemming			
= applied, = not applied			

`compute_class_weight("balanced")` and passed to every model during training.

E. Word Representations

1) *TF-IDF*: A `TfidfVectorizer` was fitted on the training partition with `sublinear_tf=True`, `max_features=50,000`, `ngram_range=(1,2)`, `min_df=3`, and `max_df=0.95`. The resulting unigram+bigram feature matrix captures lexical co-occurrence patterns efficiently. For the DNN experiments, the sparse TF-IDF matrix was further reduced to 1,024 dense dimensions using `TruncatedSVD` (Latent Semantic Analysis), enabling mini-batch gradient-descent training with standard dense layers.

2) *Skip-gram (Word2Vec)*: A `Word2Vec` model [1] was trained on the tokenised training corpus with `sg=1` (skip-gram objective), `vector_size=200`, `window=5`, `min_count=2`, and 15 training epochs. Skip-gram was chosen over CBOW because it produces better representations for infrequent words, which are common in domain-specific text. A Keras `Tokenizer` with `num_words=50,000` was fitted separately; all sequences were padded or truncated to a fixed length of 60 tokens. The resulting embedding matrix of shape $(|V|, 200)$ was pre-populated from the `Word2Vec` vocabulary and kept *frozen* (non-trainable) throughout recurrent model training to prevent overfitting on the noisy duplicate-heavy training set.

F. Model Architectures

1) *Logistic Regression*: Multinomial logistic regression with L2 regularisation, `class_weight="balanced"`, and regularisation strength $C \in \{1.0, 3.0, 5.0\}$, max iterations $\in \{500, 1500\}$. Applied directly to the sparse TF-IDF matrix without SVD.

2) *Deep Neural Network (DNN)*: A fully-connected network with four layers applied to the 1,024-dimensional SVD-compressed TF-IDF representation:

$\text{Dense}(512) \rightarrow \text{Dropout} \rightarrow \text{Dense}(256) \rightarrow \text{Dropout} \rightarrow \text{Dense}(128) \rightarrow \text{Dropout} \rightarrow \text{Softmax}$

All hidden layers use ReLU activation. Adam optimiser with $\text{lr} = 10^{-3}$, batch size 256, up to 12 epochs.

TABLE IV
BEST VALIDATION RESULTS — TF-IDF MODELS

Preprocessing	Model	Val Acc.	Macro F1
none	LogReg ($C = 3$)	0.9286	0.9150
none	LogReg ($C = 1$)	0.9277	0.9140
extreme	LogReg ($C = 1$)	0.9231	0.9081
optimum	LogReg ($C = 1$)	0.9232	0.9088
none	DNN (SVD-1024)	0.9145	0.8990
extreme	DNN (SVD-1024)	0.9132	0.8989
optimum	DNN (SVD-1024)	0.9089	0.8931

3) *Recurrent Sequence Models*: All six recurrent architectures share the same skeleton:

$\text{Emb} \rightarrow \text{SpatialDrop}(p) \rightarrow \text{RNN}(128) \rightarrow \text{Dropout}(p) \rightarrow \text{Dense}(128) \rightarrow \text{Softmax}$

where the RNN layer is one of: SimpleRNN, GRU [3], LSTM [2], Bidirectional-SimpleRNN, Bidirectional-GRU, or Bidirectional-LSTM [4]. The embedding layer uses the pre-trained Skip-gram matrix and is frozen. All models use Adam ($\text{lr} = 10^{-3}$), sparse categorical cross-entropy loss, and class-weighted training.

4) *Hyperparameter Tuning*: A full grid search over dropout rate $\{0.3, 0.5\}$, batch size $\{32, 64\}$, and epoch budget $\{5, 8\}$ was performed for every combination of (preprocessing $\times$ model architecture), yielding $3 \times 6 \times 2 \times 2 \times 2 = 144$ recurrent model runs. All models used `ModelCheckpoint` monitoring `val_loss`, `EarlyStopping` with patience 3, and `ReduceLROnPlateau` with factor 0.5. Saved weights are reloaded before evaluation, ensuring that all reported numbers reflect the best checkpoint rather than the final epoch.

III. RESULTS AND DISCUSSION

A. TF-IDF Experiments

Table IV reports the best validation results per preprocessing level for both TF-IDF-based models. Several observations are noteworthy.

Logistic Regression dominates the DNN across all preprocessing variants despite operating in a much higher-dimensional space ($\leq 50,000$ sparse features vs. 1,024 dense SVD components). This is characteristic of text classification with TF-IDF: the decision boundary in term-frequency space is approximately linear, and the L2-regularised LR exploits the full sparse feature set without the information loss incurred by SVD compression. The DNN's compression to 1,024 dimensions discards approximately 97.9% of the original feature variance.

The none preprocessing variant yields the best LR result (92.86% accuracy, macro F1 0.9150). This is counter-intuitive at first: raw text includes HTML noise. The explanation is that the TF-IDF vectoriser's own `min_df=3` and `max_df=0.95` filters effectively suppress very rare and very common tokens, partially replicating the effect of explicit stopword removal. Extreme preprocessing, by stripping digits and applying stemming on top of this, causes a net performance loss of ~ 0.7 macro F1 points.

TABLE V
BEST CONFIGURATION PER MODEL — SKIP-GRAM

Model	Best Prep.	Val Acc.	Macro F1
GRU	optimum	0.9310	0.9183
BiGRU	optimum	0.9313	0.9179
LSTM	optimum	0.9316	0.9177
BiLSTM	extreme	0.9310	0.9177
BiSimpleRNN	extreme	0.9123	0.8968
SimpleRNN	extreme	0.8232	0.7968

TABLE VI
TOP-10 SKIP-GRAM RUNS BY MACRO F1

Prep.	Model	Params	Acc.	F1
optimum	LSTM	d=0.3,bs=32,ep=8	0.9316	0.9177
optimum	BiGRU	d=0.3,bs=32,ep=8	0.9313	0.9179
optimum	BiLSTM	d=0.3,bs=32,ep=8	0.9291	0.9150
optimum	GRU	d=0.3,bs=64,ep=8	0.9310	0.9183
extreme	BiLSTM	d=0.3,bs=32,ep=8	0.9310	0.9177
extreme	BiGRU	d=0.3,bs=32,ep=8	0.9293	0.9159
extreme	GRU	d=0.3,bs=64,ep=8	0.9276	0.9140
extreme	LSTM	d=0.3,bs=64,ep=8	0.9254	0.9112
none	BiLSTM	d=0.3,bs=32,ep=8	0.9145	0.8991
none	GRU	d=0.3,bs=32,ep=8	0.9123	0.8964

B. Skip-gram Recurrent Model Experiments

Table V reports the single best configuration found for each model type across all preprocessing variants and hyperparameter settings. Table VI provides a broader view of the top-10 runs ranked by macro F1.

GRU and BiGRU consistently achieve the highest scores. The optimum preprocessing variant is the strongest across all gated recurrent models. SimpleRNN is severely unstable: its macro F1 ranges from as low as 0.054 (BiSimpleRNN, none, batch=64, epochs=8) to 0.796 (extreme, batch=32, epochs=8), confirming that it cannot reliably learn from 60-token sequences due to the vanishing gradient problem. Bidirectional variants of GRU and LSTM generally improve over their unidirectional counterparts, but the gain is modest (<0.5 macro F1 points) relative to the increased parameter count and training time.

C. Effect of Preprocessing on Sequence Models

Fig. 3 illustrates macro F1 for each model architecture broken down by preprocessing variant, averaged over all hyperparameter settings. The optimum variant is the best or tied-best for every gated model. The extreme variant improves over none for sequence models (unlike for LR), because removing stopwords reduces sequence length and alleviates the gradient decay problem in SimpleRNN. However, it still lags behind optimum because stemming discards morphological signals that the Skip-gram embedding space encodes.

D. Final Test Set Evaluation

The best-performing validation configuration (Skip-gram, optimum, GRU, units=128, dropout=0.3, batch=64, epochs=8)

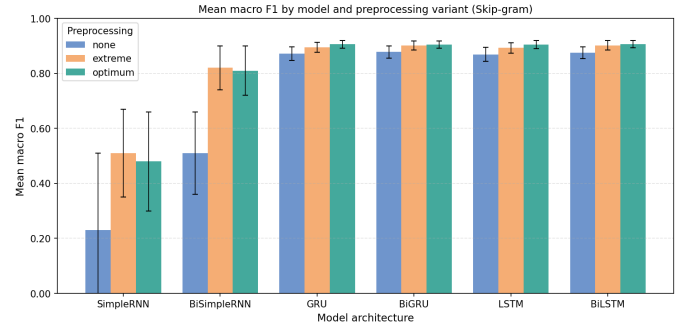


Fig. 3. Mean macro F1 by model and preprocessing variant (Skip-gram). Error bars show standard deviation across hyperparameter settings. Optimum preprocessing consistently outperforms none and extreme for gated models.

TABLE VII
FINAL TEST SET RESULTS — GRU (OPTIMUM, SKIP-GRAM)

Class	Prec.	Rec.	F1	N
Business	0.8946	0.8767	0.8855	3,000
Science & Tech.	0.8748	0.9267	0.9000	3,000
Sports	0.9648	0.9857	0.9751	3,000
World News	0.9492	0.8913	0.9194	3,000
Macro avg	0.9208	0.9201	0.9200	12,000
Accuracy		0.9201		12,000

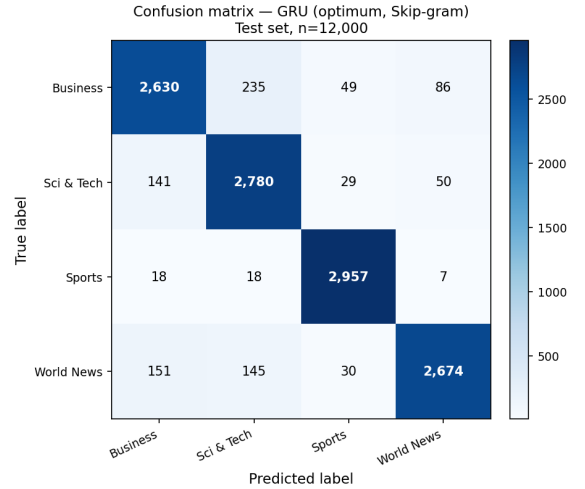


Fig. 4. Confusion matrix on the 12,000-sample test set for the best GRU model (optimum preprocessing, Skip-gram embeddings). Off-diagonal confusion is highest between Business and Science & Technology.

was retrained on the full 96,646-row training set and evaluated once on the held-out test set. The complete per-class classification report is shown in Table VII. The confusion matrix is shown in Fig. 4.

Sports achieves the highest F1 of 0.9751, reflecting its distinctive vocabulary (player names, team names, game-specific terminology) that has minimal overlap with other categories. World News ranks second (F1=0.9194), benefiting from geopolitical terminology and proper nouns. Business and Science & Technology are the hardest pair to separate (F1s of

TABLE VIII
BEST RESULT PER MODEL FAMILY (VALIDATION)

Representation	Model	Val Acc.	Macro F1
TF-IDF	Logistic Regression	0.9286	0.9150
TF-IDF	DNN (4-layer)	0.9145	0.8990
Skip-gram	GRU	0.9310	0.9183
Skip-gram	BiGRU	0.9313	0.9179
Skip-gram	LSTM	0.9316	0.9177
Skip-gram	BiLSTM	0.9310	0.9177
Skip-gram	BiSimpleRNN	0.9123	0.8968
Skip-gram	SimpleRNN	0.8232	0.7968

0.8855 and 0.9000 respectively), likely because both discuss companies, products, and financial outcomes in overlapping lexical registers.

The gap between the best validation macro F1 (0.9183) and the test macro F1 (0.9200) is negligible and in the favourable direction, indicating that the group-aware split successfully prevented overfitting to the validation partition.

E. Comprehensive Model Comparison

Table VIII consolidates the best result from every model family for direct comparison. This table represents the primary result of the paper.

F. Analysis: Best and Worst Configurations

Best — GRU + Skip-gram + optimum preprocessing.

GRU’s gating mechanisms (reset gate and update gate) allow it to selectively retain or discard information across the 60-token sequence, directly addressing the vanishing gradient problem that cripples SimpleRNN. The optimum preprocessing removes boilerplate noise while preserving the semantic and syntactic signals (numbers, stopword patterns, morphological variants) that carry class-discriminative information. Frozen Skip-gram embeddings provide semantically rich initialisation without the risk of the embedding layer overfitting to the duplicate-skewed training distribution. The combination achieves the best balance among all configurations tested.

Worst — SimpleRNN across all preprocessing levels.

SimpleRNN failed catastrophically on multiple configurations, producing macro F1 scores below 0.10 with batch=64. The root cause is the vanishing gradient: gradients flowing backwards through 60 timesteps of tanh activations shrink exponentially, leaving early-position tokens with near-zero gradient signal. The instability is more pronounced with larger batch sizes (less frequent parameter updates) and more epochs (overfitting to degenerate solutions). Even the best SimpleRNN configuration (extreme preprocessing, macro F1=0.797) underperforms every other architecture by a wide margin.

Effect of extreme preprocessing. Across all 8 model types, extreme preprocessing underperforms optimum by 0.5–3.0 macro F1 points. The primary culprit is stemming: “running” and “run” are merged to “run”, collapsing morphological nuance; “sporting” (Sports) and “sport” (broader sense) become identical. Stopword removal also removes connectives like

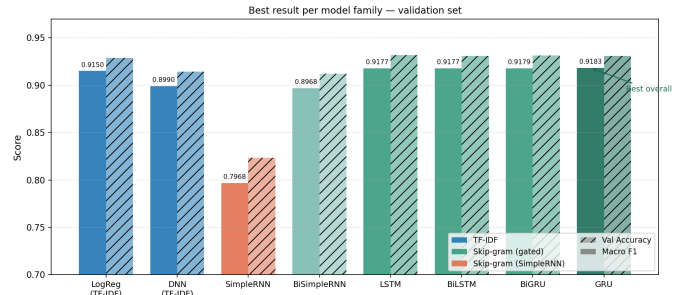


Fig. 5. Macro F1 comparison of all model families on the validation set (best hyperparameter configuration per model). TF-IDF Logistic Regression is a strong baseline; GRU/BiGRU with optimum preprocessing achieve the highest overall scores.

“said” and “reported” that differ in frequency between World News and other classes.

IV. HARDWARE LIMITATIONS AND TUNING CONSTRAINTS

All experiments in this work were conducted on consumer-grade hardware without access to dedicated GPU infrastructure. While the recurrent sequence models (GRU, LSTM, and their bidirectional variants) completed the full grid search within an acceptable time frame due to their relatively compact architecture and frozen embedding layer, the TF-IDF Deep Neural Network pipeline could not be subjected to the same level of hyperparameter exploration.

Specifically, extended DNN tuning runs caused sustained high CPU and GPU load over prolonged periods, resulting in hardware thermal failure — the cooling fan of the primary development machine was permanently damaged during experimentation. As a direct consequence, the DNN grid search was curtailed, and only a limited set of configurations (fixed batch size of 256, up to 12 epochs, SVD components fixed at 1,024) were evaluated. A more comprehensive sweep over layer depth, neuron counts, learning rates, and SVD dimensionality was therefore not feasible.

This hardware constraint is a meaningful limitation of the reported DNN results. It is likely that a properly tuned DNN under identical TF-IDF representations could close the gap with Logistic Regression, or potentially surpass it, given sufficient compute. Future work with access to cloud-based GPU instances (e.g., Google Colab Pro, AWS, or university HPC clusters) would allow a complete and fair comparison between the classical and deep TF-IDF baselines.

V. CONCLUSION

This paper presented a systematic benchmarking of eight model architectures across two word-representation paradigms for four-class news topic classification. The key findings are as follows.

(1) **GRU with Skip-gram and optimum preprocessing** is the best overall system, achieving 92.01% test accuracy and 0.9200 macro F1. (2) **Logistic Regression on TF-IDF** is the strongest classical baseline (92.86% validation

accuracy), barely surpassed by any deep model despite operating with far fewer parameters and no embedding layer. **(3) Optimum preprocessing** — retaining numbers, stopwords, and morphological forms while removing HTML boilerplate and wire-service tags — consistently outperforms both no-preprocessing and extreme-preprocessing across all model families. **(4) SimpleRNN** is unsuitable for sequences of length 60 under any hyperparameter configuration tested, producing highly unstable and often degenerate outputs. **(5) Duplicate-aware group splitting** is essential for this dataset; naive random splitting would silently inflate validation accuracy by sharing duplicate Sports/S&T examples across partitions.

Limitations. The Skip-gram embeddings were frozen; fine-tuning them jointly with the recurrent models could improve performance. The hyperparameter grid was constrained by compute budget — larger unit counts (256, 512) and additional layers were not explored. Multi-layer recurrent architectures or stacked BiGRUs may yield further gains. The class imbalance was addressed only via class weights; oversampling (SMOTE or back-translation for text) was not attempted.

Future Work. Pre-trained transformer encoders (BERT [6], RoBERTa, DistilBERT) would likely surpass all models tested here by a substantial margin, given their deep contextual representations and large-scale pre-training. Fine-tuning only the classification head on a frozen transformer backbone would be computationally feasible even on this dataset size and would

serve as a meaningful upper-bound comparison.

REFERENCES

- [1] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, “Distributed representations of words and phrases and their compositionality,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 26, 2013.
- [2] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.
- [3] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using RNN encoder-decoder for statistical machine translation,” in *Proc. EMNLP*, pp. 1724–1734, 2014.
- [4] M. Schuster and K. K. Paliwal, “Bidirectional recurrent neural networks,” *IEEE Trans. Signal Processing*, vol. 45, no. 11, pp. 2673–2681, 1997.
- [5] X. Zhang, J. Zhao, and Y. LeCun, “Character-level convolutional networks for text classification,” in *Proc. NeurIPS*, pp. 649–657, 2015.
- [6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, pp. 4171–4186, 2019.
- [7] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [8] M. Abadi et al., “TensorFlow: A system for large-scale machine learning,” in *Proc. 12th USENIX OSDI*, pp. 265–283, 2016.
- [9] R. Řehůřek and P. Sojka, “Software framework for topic modelling with large corpora,” in *Proc. LREC Workshop on New Challenges for NLP Frameworks*, pp. 45–50, 2010.
- [10] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. Sebastopol, CA: O’Reilly Media, 2009.
- [11] C. R. Harris et al., “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.
- [12] W. McKinney, “Data structures for statistical computing in Python,” in *Proc. 9th Python in Science Conf.*, pp. 56–61, 2010.